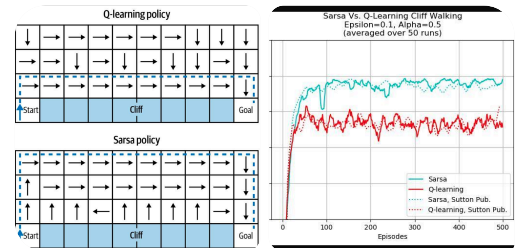


CH DRL - Q-learning與SARSA比較



一、作業目的

本作業旨在實作並比較兩種經典強化學習演算法——Q-learning 與 SARSA，透過相同環境與參數設定，分析其學習行為、收斂特性以及最終策略差異。

二、環境描述

本實驗採用經典的格子世界 (Gridworld) 問題，亦稱為「Cliff Walking」環境。環境設定如下：

使用一個矩形網格 (例如 4×12)

起點 (Start) 位於左下角

終點 (Goal) 位於右下角

起點與終點之間的底部區域為「懸崖 (Cliff)」

當代理 (agent) 進入懸崖時，會受到重大懲罰並回到起點。

三、問題設定

狀態空間 (State Space)：所有網格位置

動作空間 (Action Space)：上、下、左、右

獎勵機制 (Reward)：

每移動一步： -1

掉入懸崖： -100 ，並回到起點

到達終點：回合結束

策略 (Policy)： ϵ -greedy (例如 $\epsilon = 0.1$)

學習率 (α)：例如 0.1

折扣因子 (γ)：例如 0.9

訓練回合數 (Episodes)：至少 500 回合

四、作業內容

(一) 演算法實作

請分別實作以下兩種演算法：

Q-learning (離策略方法, Off-policy)

SARSA (同策略方法, On-policy)

兩者皆需建立並更新狀態-動作價值函數 $Q(s, a)$ 。

(二) 訓練過程

在相同的環境與參數設定下，使用 ϵ -greedy 策略進行訓練，確保兩種方法的公平比較。

(三) 結果分析

請針對以下面向進行比較與討論：

學習表現

繪製每一回合的累積獎勵 (Total Reward) 曲線

比較收斂速度

策略行為

描述或視覺化最終學習到的路徑

分析是否傾向冒險或保守

穩定性分析

比較學習過程中的波動程度

討論探索 (exploration) 對結果的影響

五、理論比較與討論

在報告中，請說明以下概念：

Q-learning 為離策略 (Off-policy) 方法，其更新基於

「下一狀態的最佳可能行動」，即使該行動未實際執行。

SARSA 為同策略 (On-policy) 方法，其更新基於「實際

採取的行動」，因此會反映探索策略的影響。

一般而言：

Q-learning 傾向學習到理論上的最優策略，但在訓練過程中可能較具風險。

SARSA 則傾向學習在實際探索策略下較安全、穩定的行為。

六、結論要求

請總結兩種方法在本實驗中的差異，並說明：

哪一種方法收斂較快

哪一種方法較穩定

在何種情境下應選擇 Q-learning 或 SARSA

這個作業要怎麼寫

這個作業可以寫成「程式實作 + 結果圖 + 理論分析」三個部分。你不用想得太複雜，核心就是：在同一個 Cliff Walking 環境中，分別用 Q-learning 和 SARSA 訓練，然後比較 reward 曲線與最後學習到的路徑。

一、報告可以這樣安排

1. 作業目的

可以這樣寫：

本作業旨在實作 Q-learning 與 SARSA 兩種強化學習演算法，並以 Cliff Walking 格子世界作為測試環境。透過相同的狀態空間、動作空間、獎勵設定與 ϵ -greedy 探索策略，比較兩種方法在學習速度、累積獎勵、策略穩定性與最終路徑選擇上的差異。

二、環境設定要寫什麼

Cliff Walking 是一個 4×12 的 Gridworld。

可以這樣寫：

環境為一個 4×12 的矩形網格，代理人從左下角 Start 出發，目標為右下角 Goal。Start 與 Goal 中間的底部格子為 Cliff 區域。若代理人走入 Cliff，會得到 -100 的懲罰，並被送回 Start；若成功到達 Goal，該 episode 結束。一般移動一步的獎勵為 -1 。

項目	設定
Grid 大小	4×12
Start	左下角
Goal	右下角
Cliff	底部中間區域
動作	上、下、左、右
一般步伐獎勵	-1
掉入懸崖	-100 並回到 Start
到達終點	episode 結束

三、兩個演算法的重點

1. Q-learning

Q-learning 是 **Off-policy**，更新時看的是下一個狀態中「最大 Q 值」的動作。

公式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

意思是：

Q-learning 不管下一步實際選了什麼動作，它都假設未來會選擇最佳動作。因此它比較容易學到最短路徑，但在訓練過程中因為還有 ϵ -greedy 探索，可能會靠近懸崖而掉下去。

2. SARSA

SARSA 是 **On-policy**，更新時用的是實際採取的下一個動作。

公式：

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

意思是：

SARSA 更新時會考慮 ϵ -greedy 實際可能做出的探索行為，所以它會知道「靠近懸崖很危險」，因此通常會學到比較保守、安全的路徑。

四、程式可以這樣寫

下面這份程式可以直接當作作業的主程式基礎。它會：

1. 建立 Cliff Walking 環境
2. 訓練 Q-learning
3. 訓練 SARSA
4. 畫出每回合累積獎勵曲線
5. 印出最後學到的 policy

如果你要畫出跟老師範例比較像的圖，建議使用：

Python

```
epsilon = 0.1
alpha = 0.5
gamma = 1.0
episodes = 500
runs = 50
```

如果老師作業明確指定 `alpha = 0.1`、`gamma = 0.9`，就改成老師指定的參數。

Python

```
import numpy as np
import matplotlib.pyplot as plt

# =====
```

```

# Environment Setting
# =====
ROWS = 4
COLS = 12

START = 3 0
GOAL = 3 11

ACTIONS =
    0  -1  0  # up
    1   1  0  # down
    2   0 -1  # left
    3   0  1  # right

ACTION_SYMBOLS =
    0  "↑"
    1  "↓"
    2  "←"
    3  "→"

def is_cliff state
    row col = state
    return row == 3 and 1 <= col <= 10

def step state action
    dr dc = ACTIONS action
    row col = state

    next_row = row + dr
    next_col = col + dc

    # Boundary check
    next_row = max 0 min ROWS - 1 next_row
    next_col = max 0 min COLS - 1 next_col

    next_state = next_row next_col

    if is_cliff next_state
        return START -100 False

    if next_state == GOAL
        return next_state -1 True

    return next_state -1 False

def state_to_index state
    row col = state
    return row * COLS + col

def epsilon_greedy Q state epsilon
    state_idx = state_to_index state

```

```

    if np.        .        < epsilon
        return np.        .        4
    else
        return np.        Q state_idx

# =====
# Q-learning
# =====
def train_q_learning episodes=500 alpha=0.5 gamma=1.0 epsilon=0.1
    Q = np.        ROWS * COLS 4
    rewards_per_episode =

    for ep in range episodes
        state = START
        done = False
        total_reward = 0

        while not done
            action = epsilon_greedy Q state epsilon
            next_state reward done = step state action

            s_idx = state_to_index state
            ns_idx = state_to_index next_state

            # Q-learning update
            Q s_idx action += alpha *
                reward + gamma * np.        Q ns_idx - Q s_idx action

            state = next_state
            total_reward += reward

        rewards_per_episode.        total_reward

    return Q rewards_per_episode

# =====
# SARSA
# =====
def train_sarsa episodes=500 alpha=0.5 gamma=1.0 epsilon=0.1
    Q = np.        ROWS * COLS 4
    rewards_per_episode =

    for ep in range episodes
        state = START
        action = epsilon_greedy Q state epsilon
        done = False
        total_reward = 0

        while not done
            next_state reward done = step state action

```

```

        s_idx = state_to_index state
        ns_idx = state_to_index next_state

        if done
            target = reward
        else
            next_action = epsilon_greedy Q next_state epsilon
            target = reward + gamma * Q ns_idx next_action

        # SARSA update
        Q s_idx action += alpha * target - Q s_idx action

        state = next_state

        if not done
            action = next_action

        total_reward += reward

    rewards_per_episode. total_reward

    return Q rewards_per_episode

# =====
# Multiple Runs Average
# =====
def average_runs algorithm runs=50 episodes=500 alpha=0.5 gamma=1.0 epsilon=0.1
    all_rewards =

    final_Q = None

    for run in range runs
        Q rewards = algorithm
        episodes=episodes
        alpha=alpha
        gamma=gamma
        epsilon=epsilon

        all_rewards. rewards
        final_Q = Q

    return final_Q np. all_rewards axis=0

# =====
# Print Policy
# =====
def print_policy Q title
    print "\n" + title

    for r in range ROWS
        row_symbols =

        for c in range COLS

```

```

        state = r c

    if state == START
        row_symbols. "S"
    elif state == GOAL
        row_symbols. "G"
    elif is_cliff state
        row_symbols. "C"
    else
        best_action = np. Q state_to_index state
        row_symbols. ACTION_SYMBOLS best_action

    print " ". row_symbols

# =====
# Main
# =====
episodes = 500
runs = 50
alpha = 0.5
gamma = 1.0
epsilon = 0.1

Q_qlearning qlearning_rewards = average_runs
    train_q_learning
    runs=runs
    episodes=episodes
    alpha=alpha
    gamma=gamma
    epsilon=epsilon

Q_sarsa sarsa_rewards = average_runs
    train_sarsa
    runs=runs
    episodes=episodes
    alpha=alpha
    gamma=gamma
    epsilon=epsilon

# Plot reward curve
plt. figsize= 10 6
plt. sarsa_rewards label="SARSA"
plt. qlearning_rewards label="Q-learning"
plt. "Episodes"
plt. "Reward Sum for Episode"
plt. "SARSA vs Q-learning in Cliff Walking"
plt.
plt. True
plt.

# Print final policies

```



```
print_policy Q_qllearning "Q-learning Policy"
print_policy Q_sarsa "SARSA Policy"
```

五、結果分析可以這樣寫

1. Reward 曲線分析

可以寫：

由實驗結果可觀察到，SARSA 的累積獎勵曲線通常高於 Q-learning，代表 SARSA 在訓練過程中掉入懸崖的次數較少，因此整體懲罰較小。Q-learning 雖然會學到接近理論最短路徑的策略，但由於訓練時仍使用 ϵ -greedy 探索，若策略太靠近懸崖，探索動作可能導致代理人掉入 Cliff，因此 reward 曲線波動較大且平均獎勵較低。

2. 最終策略分析

可以寫：

Q-learning 最終學到的策略通常會選擇靠近 Cliff 的最短路徑，因為它更新時假設下一步會採取最佳行動，因此傾向追求理論上的最佳路徑。然而，這種路徑在仍有探索行為時具有較高風險。

SARSA 則會學到離 Cliff 較遠的安全路徑，因為它更新時考慮的是 ϵ -greedy 實際採取的下一個動作。也就是說，SARSA 會把探索可能造成的危險也納入學習，因此策略較保守但較穩定。

六、Q-learning 和 SARSA 差異表

比較項目	Q-learning	SARSA
方法類型	Off-policy	On-policy
更新依據	下一狀態的最大 Q 值	下一狀態實際採取的動作
策略傾向	較冒險	較保守
Cliff Walking 路徑	靠近懸崖的最短路徑	遠離懸崖的安全路徑
訓練穩定性	較容易波動	較穩定

比較項目	Q-learning	SARSA
平均 reward	可能較低，因為容易掉入懸崖	通常較高
適合情境	已知環境、追求最短最佳策略	探索風險高、需要安全學習

七、結論可以這樣寫

可以直接放進報告：

本實驗比較 Q-learning 與 SARSA 在 Cliff Walking 環境中的學習表現。結果顯示，Q-learning 因為採用 off-policy 更新方式，會根據下一狀態的最大 Q 值進行學習，因此傾向學到靠近懸崖的最短路徑。雖然此策略在理論上較接近最佳解，但在 ϵ -greedy 探索過程中，代理人容易因隨機動作而掉入懸崖，導致累積獎勵波動較大。

相較之下，SARSA 屬於 on-policy 方法，其更新會考慮實際採取的下一個動作，因此能反映探索策略帶來的風險。在 Cliff Walking 問題中，SARSA 通常會學到較安全、遠離懸崖的路徑，因此訓練過程較穩定，平均累積獎勵也較高。

整體而言，若目標是追求理論上的最短最佳路徑，可以選擇 Q-learning；若環境中探索行為具有較高風險，或希望訓練過程更加穩定，則 SARSA 會是較合適的方法。

你這份作業最後應該交的東西大概是：

1. 程式碼
2. reward 曲線圖
3. Q-learning 和 SARSA 的 final policy
4. 兩者差異分析與結論

最重要的是要能解釋出這句話：

Q-learning 比較像「學理論上的最佳路徑」，SARSA 比較像「學實際探索時比較安全的路徑」。

y軸的範圍可以跟圖片一樣-100 ~ 0嗎

可以，可以直接把 y 軸範圍固定成 -100 ~ 0。

你在畫圖那段加上這一行就好：

<> Python

```
plt.      -100  0
```

例如把原本畫圖改成：

<> Python

```
plt.      figsize= 10  6
plt.      sarsa_rewards  label="SARSA"
plt.      qlearning_rewards  label="Q-learning"
plt.      "Episodes"
plt.      "Reward Sum for Episode"
plt.      "SARSA vs Q-learning in Cliff Walking"
plt.
plt.      True

plt.      -100  0      # 固定 y 軸範圍
plt.      0  500      # 如果你也想讓 x 軸跟圖一樣
plt.
```

如果你想更像老師那張圖

可以再加上：

<> Python

```
plt.      np.      -100  1  20
```

完整會變成：

<> Python

```
plt.      figsize= 10  6
plt.      sarsa_rewards  label="SARSA"
plt.      qlearning_rewards  label="Q-learning"
plt.      "Episodes"
plt.      "Reward Sum for Episode"
plt.      "SARSA vs Q-learning in Cliff Walking"
plt.
plt.      True

plt.      0  500
plt.      -100  0
plt.      np.      -100  1  20

plt.
```

要注意一件事

如果某些 episode 的總 reward 小於 -100，那曲線超過的部分會被切掉，看不到。
但這其實是正常的，因為你只是把顯示範圍固定成和範例圖一樣。

如果你要，我也可以直接幫你把 整份 Cliff Walking 程式碼改成和老師圖風格更接近的版本。

可以實線是mean虛線是var嗎

可以，但我要先提醒你一個重點：

先注意

你前面想把 y 軸固定成 `-100 ~ 0`，
可是如果你畫的是 **variance** (變異數)，那它一定是 ≥ 0 的。

所以：

- **mean reward**：通常在 `-100 ~ 0`
- **variance**：通常在 `0` 以上

這樣如果你把 y 軸固定成 `-100 ~ 0`，虛線的 **variance** 幾乎會看不到。

如果你真的想要：

- 實線 = mean
- 虛線 = variance

那比較建議用 **雙 y 軸**，這樣最清楚。

一、先把平均和變異數都算出來

把你原本的 `average_runs()` 改成這樣：

```
<> Python
```

```
def statistics_runs algorithm runs=50 episodes=500 alpha=0.5 gamma=1.0 epsilon=0.1
    all_rewards =
    final_Q = None

    for run in range runs
        Q rewards = algorithm
        episodes=episodes
        alpha=alpha
        gamma=gamma
        epsilon=epsilon

        all_rewards. rewards
        final_Q = Q

    all_rewards = np. all_rewards # shape: (runs, episodes)
    mean_rewards = np. all_rewards axis=0
    var_rewards = np. all_rewards axis=0

    return final_Q mean_rewards var_rewards
```

二、主程式改成這樣

Python

```
Q_qlearning qlearning_mean qlearning_var = statistics_runs
    train_q_learning
    runs=runs
    episodes=episodes
    alpha=alpha
    gamma=gamma
    epsilon=epsilon

Q_sarsa sarsa_mean sarsa_var = statistics_runs
    train_sarsa
    runs=runs
    episodes=episodes
    alpha=alpha
    gamma=gamma
    epsilon=epsilon
```

三、畫圖：實線是 mean，虛線是 variance (雙 y 軸版本)

Python

```
plt. figsize= 10 6
ax1 = plt.
```

```

# 左邊 y 軸: mean
ax1.      sarsa_mean  label="Sarsa Mean"  linestyle='-'
ax1.      qlearning_mean  label="Q-learning Mean"  linestyle='-'
ax1.      "Episodes"
ax1.      "Mean Reward"
ax1.      0 500
ax1.      -100 0
ax1.      np.      -100 1 20
ax1.      True

# 右邊 y 軸: variance
ax2 = ax1.
ax2.      sarsa_var  label="Sarsa Variance"  linestyle='--'
ax2.      qlearning_var  label="Q-learning Variance"  linestyle='--'
ax2.      "Variance"

# 合併圖例
lines1  labels1 = ax1.
lines2  labels2 = ax2.
ax1.      lines1 + lines2  labels1 + labels2  loc="best"

plt.      "Sarsa vs Q-learning Cliff Walking\nSolid = Mean, Dashed = Variance"
plt.

```

如果你想全部畫在同一個 y 軸

也可以，但我不太建議，因為 mean 和 variance 的數值尺度不同，看起來會怪怪的。

Python

```

plt.      figsize= 10 6

plt.      sarsa_mean  label="Sarsa Mean"  linestyle='-'
plt.      qlearning_mean  label="Q-learning Mean"  linestyle='-'
plt.      sarsa_var  label="Sarsa Variance"  linestyle='--'
plt.      qlearning_var  label="Q-learning Variance"  linestyle='--'

plt.      "Episodes"
plt.      "Value"
plt.      "Sarsa vs Q-learning"
plt.
plt.      True
plt.

```

但這種情況下，你就不要再固定 `ylim(-100, 0)`，不然 var 會被截掉。

如果你其實想表達「波動程度」

很多老師其實更常接受的是：

- 實線 = mean
- 虛線 = mean $\pm$ std

這樣圖會更像「平均趨勢 + 波動範圍」，而且還能維持和 reward 同一個尺度。

例如：

Python

```
qlearning_std = np.      qlearning_var
sarsa_std = np.      sarsa_var

plt.      figsize= 10 6

plt.      sarsa_mean label="Sarsa Mean" linestyle='-'
plt.      qlearning_mean label="Q-learning Mean" linestyle='-'

plt.      sarsa_mean + sarsa_std label="Sarsa Mean + Std" linestyle='--'
plt.      sarsa_mean - sarsa_std label="Sarsa Mean - Std" linestyle='--'

plt.      qlearning_mean + qlearning_std label="Q-learning Mean + Std" linestyle='--'
plt.      qlearning_mean - qlearning_std label="Q-learning Mean - Std" linestyle='--'

plt.      "Episodes"
plt.      "Reward Sum for Episode"
plt.      "Sarsa vs Q-learning Cliff Walking"
plt.      0 500
plt.      -100 0
plt.      np.      -100 1 20
plt.
plt.      True
plt.
```

我建議你怎麼做

如果你是要交作業，我建議：

最好版本

- 實線：mean reward
- 陰影或虛線：std / variability
- 或者 雙 y 軸：左邊 mean，右邊 variance

因為這樣最合理。

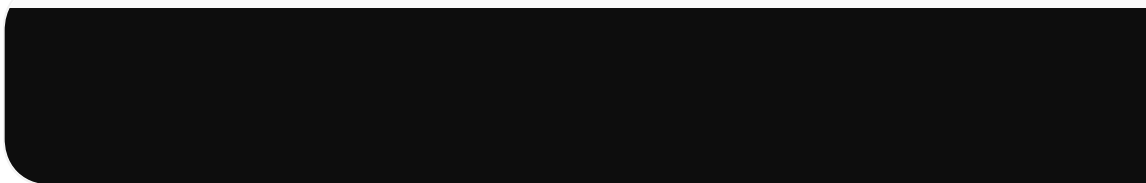
如果你要，我下一則可以直接幫你整理成一份 **可直接執行的完整 Cliff Walking 程式碼**，裡面幫你做好：

1. Q-learning
2. SARSA
3. 多次 run
4. mean / variance 計算
5. 圖畫成「實線 mean、虛線 variance」版本。

要不然老師圖裡的版本的虛線代表什麼

老師圖裡的**虛線不是 variance (變異數)**。

從圖例可以看到：



所以它的意思是：

線條	代表意思
實線 Sarsa	自己程式跑出來的 SARSA 平均 reward
實線 Q-learning	自己程式跑出來的 Q- learning 平均 reward
虛線 Sarsa, Sutton Pub.	Sutton 強化學習教材或 範例中的 SARSA 參考結 果
虛線 Q-learning, Sutton Pub.	Sutton 強化學習教材或 範例中的 Q-learning 參 考結果

也就是說，老師圖中的虛線是**參考文獻中的結果曲線**，用來跟自己實作的結果比較，不是標準差，也不是變異數。

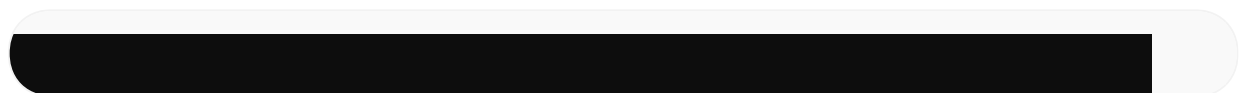
為什麼虛線和實線很像？

因為兩者都是在同一個 Cliff Walking 環境下，用類似參數跑出來的平均結果，例如：

Python

```
epsilon = 0.1
alpha = 0.5
gamma = 1.0
episodes = 500
runs = 50
```

所以老師的圖是在比較：



那你的作業要不要畫虛線？

如果老師沒有給你 Sutton publication 的原始數據，你可以不用畫虛線。

你的圖只要畫：

Python

```
plt.      sarsa_rewards  label="SARSA"
plt.      qlearning_rewards  label="Q-learning"
```

就可以。

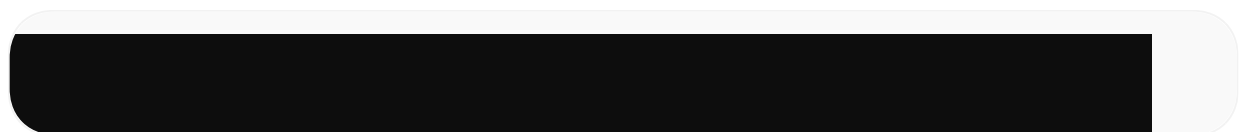
如果要模仿老師圖的說法，可以在報告中寫：

本實驗僅繪製自行實作之 Q-learning 與 SARSA 的平均累積獎勵曲線；圖中未加入 Sutton publication 的參考曲線，因為未提供原始參考數據。

如果你想分析穩定性

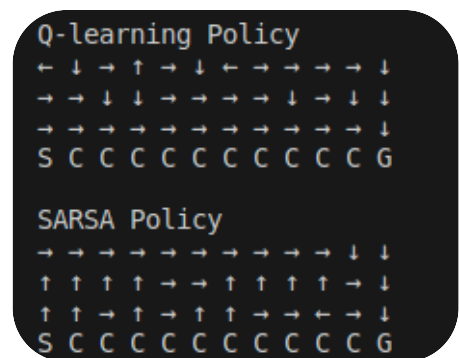
穩定性不要用老師圖的虛線來解釋，因為那不是 variance。

你可以另外寫：



簡單講：

老師圖的虛線 = Sutton 教材的參考結果，不是 variance。



所以這個policy就可以看表達出這個嗎：
描述或視覺化最終學習到的路徑
分析是否傾向冒險或保守

對，可以表達，而且其實已經能看出重點了：

1. 描述或視覺化最終學習到的路徑
2. 分析策略偏冒險還是保守

不過要更精確地說：

- 你這張 policy 圖 表示的是
「每個 state 最佳動作是什麼」
- 它可以用來推論最終路徑
- 但如果老師要你更明確「畫出路徑」，最好再補一個
從 Start 出發，依照 policy 一路走到 Goal 的實際路徑圖
或至少用文字說明路徑怎麼走

一、你的這張 policy 已經能看出什麼？

1. Q-learning Policy

從圖上看：

- Start 在左下角
- 它先往上走一格
- 然後幾乎沿著 懸崖上方那一系列 一直往右
- 最後在靠近 Goal 時往下走

這代表：

- 它學到的是 較短的路徑
- 但是這條路 非常靠近 Cliff
- 如果 ϵ -greedy 還有探索，稍微走錯就可能掉下去

所以你可以分析成：

Q-learning 學到的策略傾向沿著懸崖邊緣前進，屬於較具冒險性的策略。雖然此路徑較短、理論上接近最優路徑，但在探索過程中較容易因隨機動作而掉入 Cliff，因此風險較高。

2. SARSA Policy

從圖上看：

- 它不是貼著懸崖走
- 會先往上走到比較安全的區域
- 再一路往右
- 最後再往下到 Goal

這代表：

- 它故意避開 Cliff
- 雖然路徑比較長
- 但更安全、穩定

所以你可以分析成：

SARSA 學到的策略會刻意遠離懸崖區域，先往上移動到較安全的位置，再向右移動至目標，因此屬於較保守的策略。雖然路徑較長，但可有效降低探索時掉入 Cliff 的風險。

二、所以這能不能回答老師要求？

老師要求 1：描述或視覺化最終學習到的路徑

可以，這張圖可以做到。

因為這張 policy 圖其實已經視覺化了每個格子的最佳動作。

只要你再補一句：

從 Start 狀態依照箭頭方向前進，即可得到最終學習到的路徑。

這樣就夠了。

老師要求 2：分析是否傾向冒險或保守

也可以，而且你的圖非常適合回答這題。

因為：

- **Q-learning**：走最短、貼近懸崖 → 冒險
- **SARSA**：繞遠、遠離懸崖 → 保守

這正是 Cliff Walking 最經典的比較結果。

三、你在報告裡可以直接這樣寫

最終策略視覺化分析

由最終學習得到的 policy 可觀察到，Q-learning 與 SARSA 在路徑選擇上具有明顯差異。

Q-learning 的策略通常會沿著懸崖上方最近的一列前進，也就是選擇從 Start 往上移動一格後，幾乎一路向右移動直到接近 Goal，再向下抵達終點。這表示 Q-learning 傾向學習理論上的最短路徑，因此具有較強的冒險性。雖然路徑長度較短，但在 ϵ -greedy 探索下，若隨機選錯動作，代理人容易掉入懸崖而受到較大懲罰。

相較之下，SARSA 的策略則明顯更保守。從 policy 可看出，代理人會先向上移動至離懸崖較遠的位置，再向右前進，最後再往下走向 Goal。雖然此路徑較長，但由於遠離 Cliff，因此能有效降低探索過程中的風險，使學習過程較為穩定。

因此，從最終 policy 可以清楚看出：Q-learning 傾向冒險，SARSA 傾向保守。

四、如果你想讓「最終路徑」更明顯，建議再補一個小功能

你現在印出的 policy 已經很好了，但如果想更完整，可以再加一個：

方法：從 Start 開始，按照 greedy policy 真正走一次

這樣你就能得到像：

- Q-learning path:
(3,0) -> (2,0) -> (2,1) -> ... -> (2,11) -> (3,11)
- SARSA path:
(3,0) -> (2,0) -> (1,0) -> (0,0) -> ... -> (0,11) -> (1,11) -> (2,11) -> (3,11)

這樣就更像「最終路徑」。

五、簡單結論

你可以這樣回答老師這題：

是的，policy 圖可以用來描述與視覺化最終學習到的路徑，因為它顯示了每個狀態下的最佳動作方向。從圖中可看出，Q-learning 傾向沿著懸崖邊緣走最短路徑，屬於較冒險的策略；而 SARSA 則會遠離懸崖、選擇較安全但較長的路徑，屬於較保守的策略。

如果你要，我下一則可以直接幫你補：

1. 一段可放報告的正式文字
2. 一個 `extract_path()` 函式，把最終路徑直接印出來
3. 把路徑畫在 grid 上的版本，讓你交作業更完整。

穩定性分析

比較學習過程中的波動程度

討論探索 (exploration) 對結果的影響

五、理論比較與討論

在報告中，請說明以下概念：

Q-learning 為離策略 (Off-policy) 方法，其更新基於「下一狀態的最佳可能行動」，即使該行動未實際執行。SARSA 為同策略 (On-policy) 方法，其更新基於「實際採取的行動」，因此會反映探索策略的影響。一般而言：

Q-learning 傾向學習到理論上的最優策略，但在訓練過程中可能較具風險。

SARSA 則傾向學習在實際探索策略下較安全、穩定的行為。

六、結論要求

請總結兩種方法在本實驗中的差異，並說明：

哪一種方法收斂較快

哪一種方法較穩定

在何種情境下應選擇 Q-learning 或 SARSA 這幾個要怎麼寫

可以，這幾段其實就是報告的**討論與結論部分**。你可以直接照下面這樣寫，再依照你自己的 reward 圖稍微修改。

四、結果分析：穩定性分析

4.3 穩定性分析

由累積獎勵曲線可以觀察到，Q-learning 與 SARSA 在學習過程中的波動程度不同。Q-learning 的 reward 曲線通常波動較明顯，且平均 reward 較低，原因是 Q-learning 最後傾向學到靠近懸崖邊緣的最短路徑。雖然這條路徑在理論上較接近最佳路徑，但在訓練過程中仍然使用 ϵ -greedy 探索策略，因此代理人有機率隨機選到錯誤動作而掉入懸崖，導致該回合得到 -100 的懲罰，使 reward 曲線出現較大的下降與震盪。

相較之下，SARSA 的 reward 曲線通常較平穩，且平均累積獎勵較高。這是因為 SARSA 屬於 on-policy 方法，更新時會考慮實際依照 ϵ -greedy 策略採取的下一個動作。因此，SARSA 在學習過程中會把探索可能造成的風險也納入考量，最後傾向學到遠離懸崖的安全路徑。雖然此路徑比 Q-learning 的路徑稍長，但較不容易因探索動作而掉入懸崖，因此整體學習過程較穩定。

探索對結果的影響

可以接在穩定性分析後面寫：

在本實驗中，探索策略使用 ϵ -greedy。此策略代表代理人在大多數情況下會選擇目前 Q 值最大的動作，但仍有 ϵ 的機率隨機選擇其他動作。探索有助於代理人嘗試不同路徑，避免過早陷入局部最佳解，但在 Cliff Walking 環境中，探索也會帶來風險。

對 Q-learning 而言，由於其學習到的策略較接近懸崖，當代理人因 ϵ -greedy 隨機探索而選到向下的動作時，就可能掉入 Cliff，造成重大懲罰。因此 Q-learning 的訓練過程較容易出現 reward 大幅下降的情況。

對 SARSA 而言，由於它的更新方式會反映實際探索策略的影響，因此會逐漸學到「靠近懸崖是危險的」。因此 SARSA 會選擇較遠離懸崖的路徑，使探索時掉入懸崖的機率降低。這也是 SARSA 在本實驗中通常較穩定的主要原因。

五、理論比較與討論

5.1 Q-learning

Q-learning 是一種離策略方法，也就是 off-policy method。其更新公式如下：

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

其中， s 表示目前狀態， a 表示目前採取的動作， r 表示獎勵， s' 表示下一個狀態， α 是學習率， γ 是折扣因子。

Q-learning 的重點是，它在更新 Q 值時使用的是下一個狀態中「最大 Q 值」的動作，也就是：

$$\max_{a'} Q(s', a')$$

這代表 Q-learning 假設代理人在下一個狀態會採取最佳行動，即使該行動在實際訓練中不一定真的被執行。因此，Q-learning 傾向學到理論上的最短或最佳路徑。在 Cliff Walking 問題中，這會使代理人學到靠近懸崖邊緣的最短路徑，因此策略較具冒險性。

5.2 SARSA

SARSA 是一種同策略方法，也就是 on-policy method。其更新公式如下：

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

SARSA 的名稱來自於更新過程中使用的五個元素：

$$S, A, R, S', A'$$

也就是目前狀態、目前動作、獎勵、下一個狀態，以及下一個實際採取的動作。

SARSA 與 Q-learning 最大的不同在於，SARSA 並不是使用下一狀態的最大 Q 值，而是使用下一個狀態中「實際被 ϵ -greedy 選到的動作」來更新。因此，SARSA 會受到目前探索策略的影響。

在 Cliff Walking 問題中，因為 ϵ -greedy 仍有機率隨機探索，若代理人太靠近懸崖，就容易因隨機動作而掉入 Cliff。SARSA 會把這種探索風險納入學習，因此會傾向選擇遠離懸崖的安全路徑。這使得 SARSA 的策略較保守，但訓練過程通常較穩定。

Q-learning 與 SARSA 比較表

比較項目	Q-learning	SARSA
方法類型	Off-policy 離策略	On-policy 同策略
更新依據	下一狀態的最大 Q 值	下一狀態實際採取的動作
是否考慮探索風險	較少考慮	會考慮
學到的路徑	較短、靠近懸崖	較長、遠離懸崖
策略特性	較冒險	較保守

比較項目	Q-learning	SARSA
reward 曲線	較容易波動	較穩定
適合情境	追求理論最佳策略	需要安全與穩定學習

六、結論

可以直接這樣寫：

本實驗在 Cliff Walking 環境中比較 Q-learning 與 SARSA 兩種強化學習演算法。實驗結果顯示，兩者皆能學習到從 Start 到 Goal 的路徑，但其學習到的策略具有明顯差異。

Q-learning 屬於 off-policy 方法，其更新時使用下一狀態的最大 Q 值，因此傾向學習理論上的最佳策略。在本實驗中，Q-learning 通常會學到沿著懸崖上方前進的最短路徑。此路徑雖然步數較少，但因為非常接近 Cliff，在 ϵ -greedy 探索過程中容易因隨機動作而掉入懸崖，因此 reward 曲線波動較大，訓練過程較不穩定。

SARSA 屬於 on-policy 方法，其更新時使用實際採取的下一個動作，因此會反映探索策略造成的影響。由於 SARSA 會考慮 ϵ -greedy 探索可能帶來的風險，因此在 Cliff Walking 環境中通常會學到較安全的路徑，也就是先遠離懸崖，再前往終點。雖然此路徑較長，但能降低掉入懸崖的機率，使學習過程較穩定。

就收斂速度而言，Q-learning 較容易學到理論上的最短路徑，因此若只觀察最終 greedy policy，Q-learning 可能較快接近最佳策略。然而，若從訓練過程中的平均累積獎勵來看，SARSA 的表現通常較穩定，且 reward 較不容易因掉入懸崖而大幅下降。

因此，若環境風險較低，且目標是追求理論上的最短或最佳策略，可以選擇 Q-learning；若環境中探索行為可能造成嚴重懲罰，或希望訓練過程更加安全穩定，則 SARSA 會是較合適的方法。

最後三個問題可以簡短回答

哪一種方法收斂較快？

可以寫：

Q-learning 通常較快學到接近理論最佳的最短路徑，因此在策略收斂上可能較快。不過，由於該路徑靠近懸崖，在訓練時容易受到探索影響而掉入 Cliff，使 reward 曲線仍可能有較大波動。

哪一種方法較穩定？

可以寫：

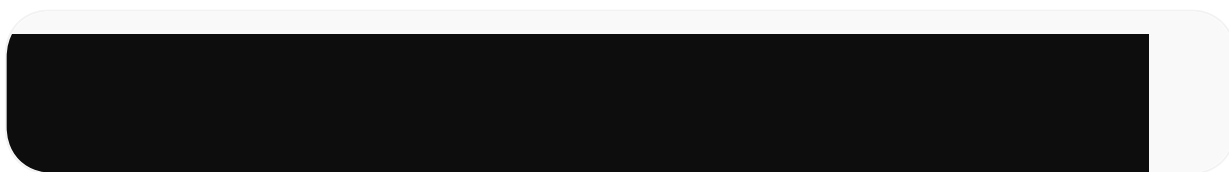
SARSA 較穩定。因為 SARSA 的更新會考慮實際 ϵ -greedy 探索策略下採取的動作，因此能反映探索可能造成的風險。在 Cliff Walking 問題中，SARSA 會學到較保守且遠離懸崖的路徑，使掉入 Cliff 的機率降低，因此 reward 曲線較平穩。

何種情境下選擇 Q-learning 或 SARSA？

可以寫：

若任務目標是追求理論上的最佳路徑，且探索過程中的風險不高，可以選擇 Q-learning。若任務環境具有較高風險，例如錯誤動作會造成嚴重懲罰，或希望訓練過程更加安全穩定，則適合選擇 SARSA。

你可以把報告這幾段整理成：



這樣就會很完整。